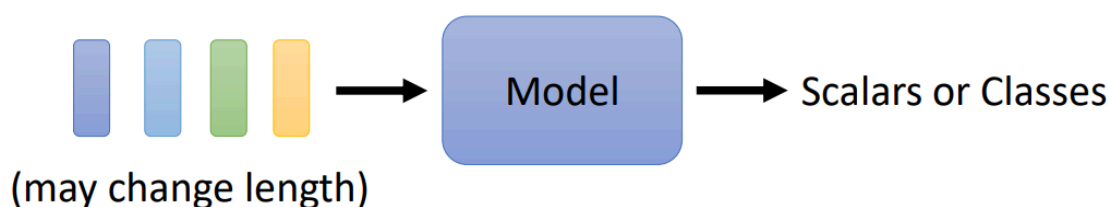


Transformer HuangYi Lee

🕒 Created time	@January 20, 2026 10:42 PM
🕒 Last edited time	@January 28, 2026 5:14 PM

Self attention

- Input is a set of vectors



序列问题

输入

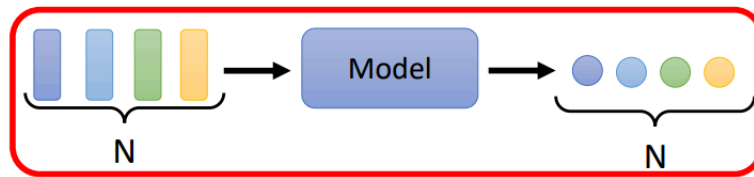
1. 文字处理
 - 输入是序列，可以用one-hot编码，但是这样就假设所有的词汇是相互之间是没有联系的（one-hot编码下，向量之间是正交的）
 - word-embedding：
2. 声音信号处理：每一段时间窗口内的信号看作一个frame，每个frame可以用一个向量表示
3. 图：社交网路图、药物分子图，每个节点可以用一个向量表示

输出

What is the output?

- Each vector has a label.

focus of this lecture

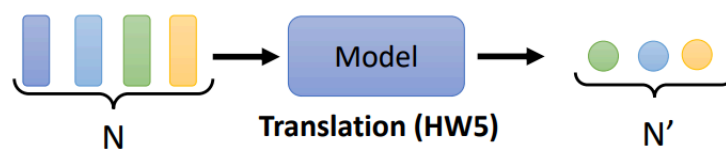


- The whole sequence has a label.



- Model decides the number of labels itself.

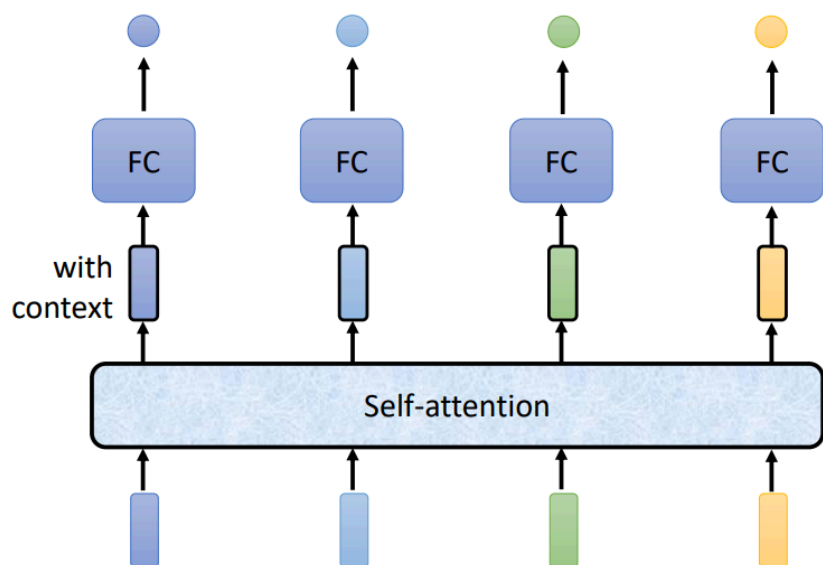
seq2seq



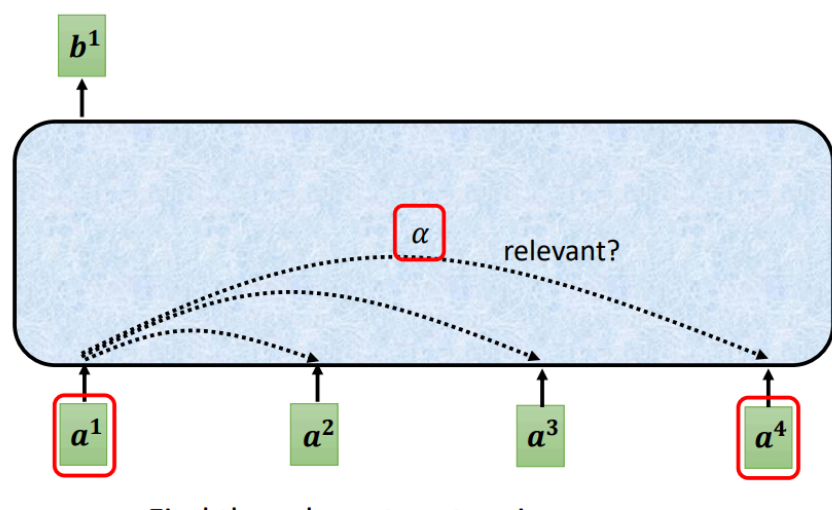
1. N对N：词性判断
2. N对1：评价打分、药物性质分析
3. N对M：模型自己决定输出长度——翻译

Sequence Labeling (N对N)

- 一种想法是对于序列的每一个输入，单独训练一个Fully-connected Network
 - 无法考虑上下文
 - 计算量过大
- 针对上一种思路，可以以窗口作为输入
 - 不好确认窗口大小
 - 窗口太大同样会导致计算量过大
- Self-attention
 - 添加一个self-attention层，向量经过这个层之后可以获取考虑了整个输入序列的上下文信息

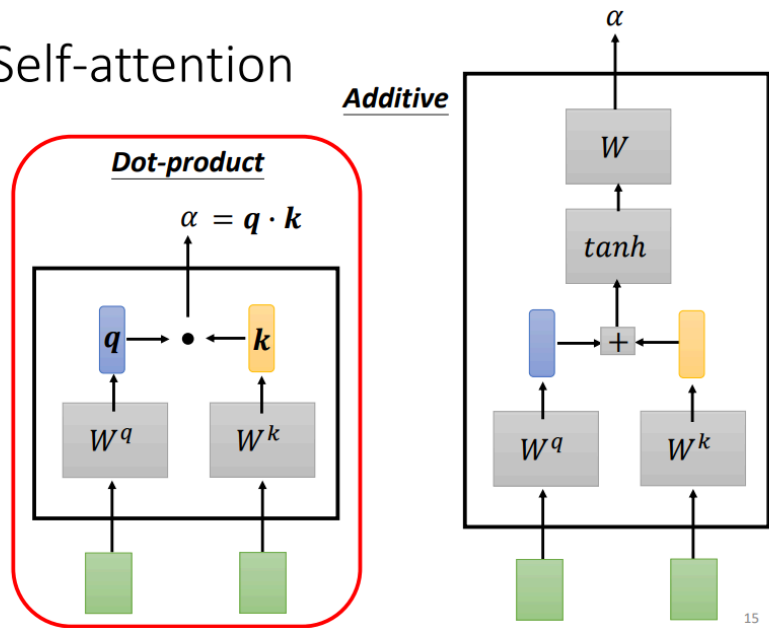


- 。 具体如何考虑整个上下文：对于当前向量，计算其与其他向量的关联性



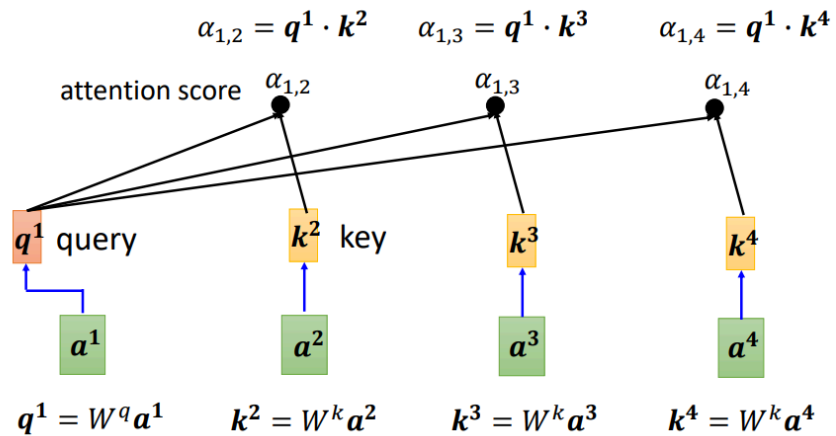
- 。 如何计算相关性：dot-product

Self-attention

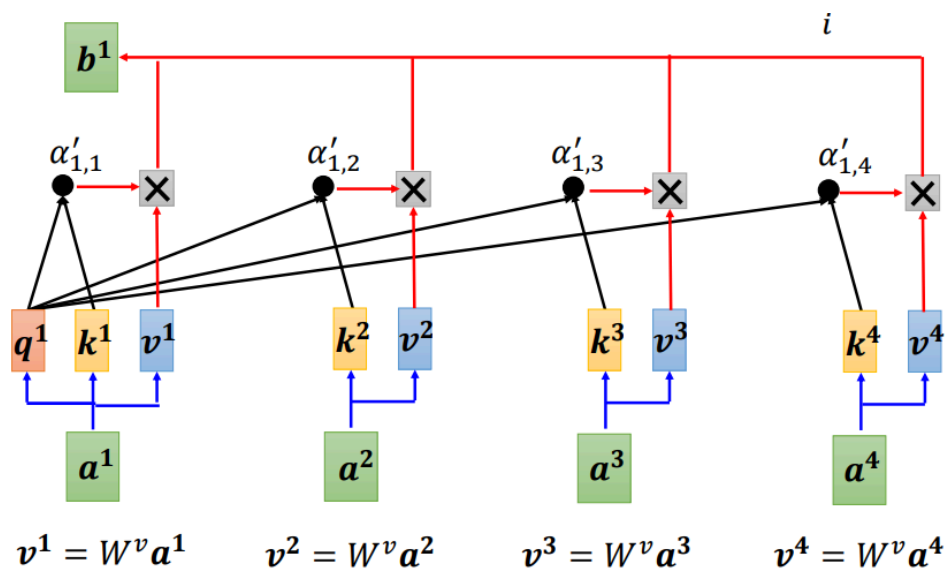


具体计算步骤：

1. 对于当前向量，乘上 W^q ，得到query
2. 对于其他所有向量，乘上 W^k ，得到key
3. q 与 k 相乘，得到attention_score分数



4. 对于每个向量，再乘上 W^v ，得到value
5. attention_score 与 value按元素相乘再相加，得到最后抽取的信息 $b^1 = \sum a'_{1,i} \cdot v^i$



- 对于Q、K、V的计算可以并行化

$$\begin{aligned}
 q^i &= W^q a^i & \begin{matrix} q^1 & q^2 & q^3 & q^4 \end{matrix} &= \begin{matrix} W^q \end{matrix} \begin{matrix} a^1 & a^2 & a^3 & a^4 \end{matrix} \\
 & & Q & & I \\
 k^i &= W^k a^i & \begin{matrix} k^1 & k^2 & k^3 & k^4 \end{matrix} &= \begin{matrix} W^k \end{matrix} \begin{matrix} a^1 & a^2 & a^3 & a^4 \end{matrix} \\
 & & K & & I \\
 v^i &= W^v a^i & \begin{matrix} v^1 & v^2 & v^3 & v^4 \end{matrix} &= \begin{matrix} W^v \end{matrix} \begin{matrix} a^1 & a^2 & a^3 & a^4 \end{matrix} \\
 & & V & & I
 \end{aligned}$$

- Attention_score的计算同样可以并行

$$\begin{matrix} \alpha'_{1,1} & \alpha'_{2,1} & \alpha'_{3,1} & \alpha'_{4,1} \\ \alpha'_{1,2} & \alpha'_{2,2} & \alpha'_{3,2} & \alpha'_{4,2} \\ \alpha'_{1,3} & \alpha'_{2,3} & \alpha'_{3,3} & \alpha'_{4,3} \\ \alpha'_{1,4} & \alpha'_{2,4} & \alpha'_{3,4} & \alpha'_{4,4} \end{matrix} \xleftarrow{\text{softmax}} \begin{matrix} \alpha_{1,1} & \alpha_{2,1} & \alpha_{3,1} & \alpha_{4,1} \\ \alpha_{1,2} & \alpha_{2,2} & \alpha_{3,2} & \alpha_{4,2} \\ \alpha_{1,3} & \alpha_{2,3} & \alpha_{3,3} & \alpha_{4,3} \\ \alpha_{1,4} & \alpha_{2,4} & \alpha_{3,4} & \alpha_{4,4} \end{matrix} = \begin{matrix} k^1 \\ k^2 \\ k^3 \\ k^4 \end{matrix} \begin{matrix} q^1 & q^2 & q^3 & q^4 \end{matrix}$$

A' A K^T Q

- 最后相关性的计算

$$\begin{array}{c}
 \boxed{b^1} \boxed{b^2} \boxed{b^3} \boxed{b^4} \\
 O
 \end{array}
 =
 \begin{array}{c}
 \boxed{v^1} \boxed{v^2} \boxed{v^3} \boxed{v^4} \\
 V
 \end{array}
 \begin{array}{c}
 \boxed{\alpha'_{1,1}} \boxed{\alpha'_{2,1}} \boxed{\alpha'_{3,1}} \boxed{\alpha'_{4,1}} \\
 \boxed{\alpha'_{1,2}} \boxed{\alpha'_{2,2}} \boxed{\alpha'_{3,2}} \boxed{\alpha'_{4,2}} \\
 \boxed{\alpha'_{1,3}} \boxed{\alpha'_{2,3}} \boxed{\alpha'_{3,3}} \boxed{\alpha'_{4,3}} \\
 \boxed{\alpha'_{1,4}} \boxed{\alpha'_{2,4}} \boxed{\alpha'_{3,4}} \boxed{\alpha'_{4,4}} \\
 A'
 \end{array}$$

所以只有 W_q, W_k, W_v 是需要学习的参数

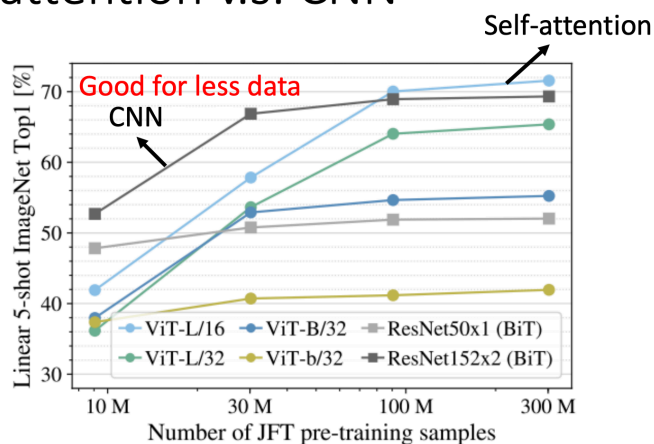


CNN就是只考虑特别感受野的self-attention

RNN相对self-attention，相隔远的向量之间联系很难记住，而且RNN无法并行计算

Self-attention v.s. CNN

Good for more data



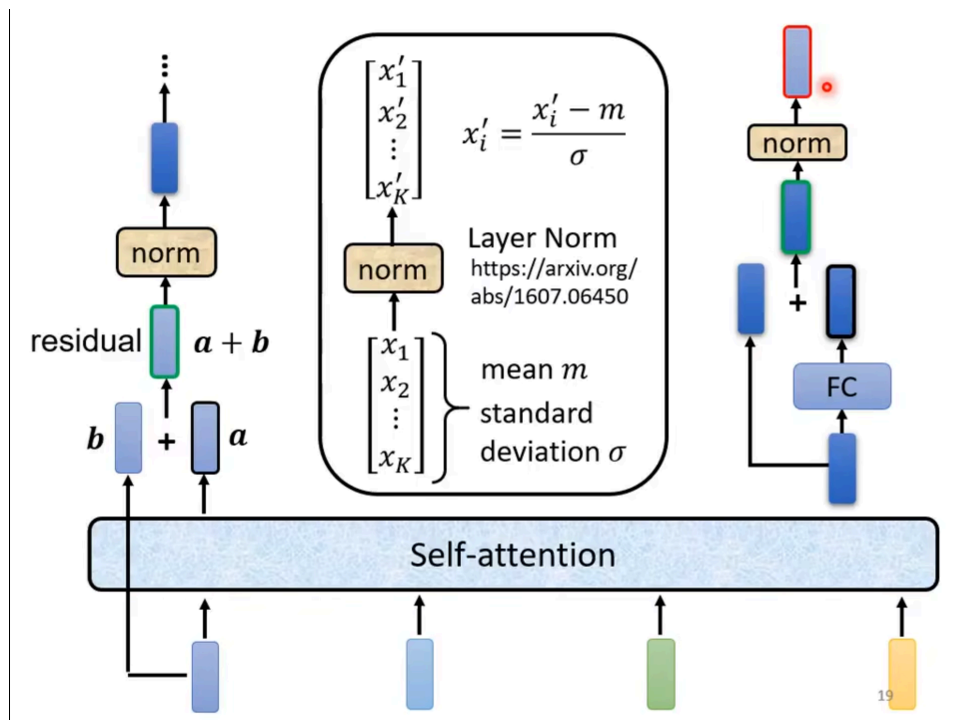
An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale

<https://arxiv.org/pdf/2010.11929.pdf>

这是ViT这个论文中对Transformer与传统CNN的比较，在图表的右上方可以看到在数据集较大时Transformer最终取得了较好的效果，但是这样的比较真的合理吗？假如把ResNet152替换成更大的CNN会不会其结果又能反超ViT呢？

Transformer

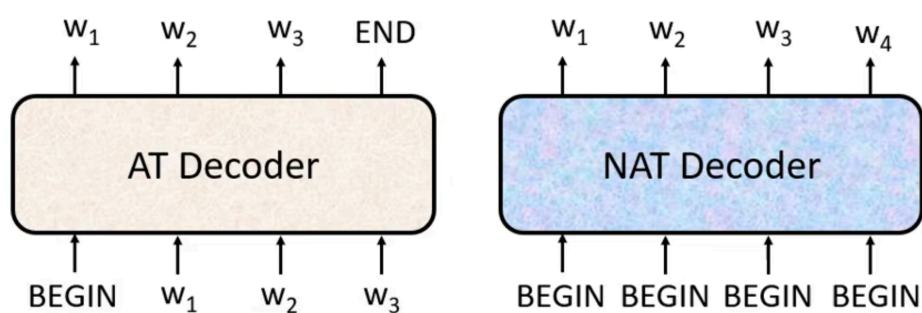
Encoder



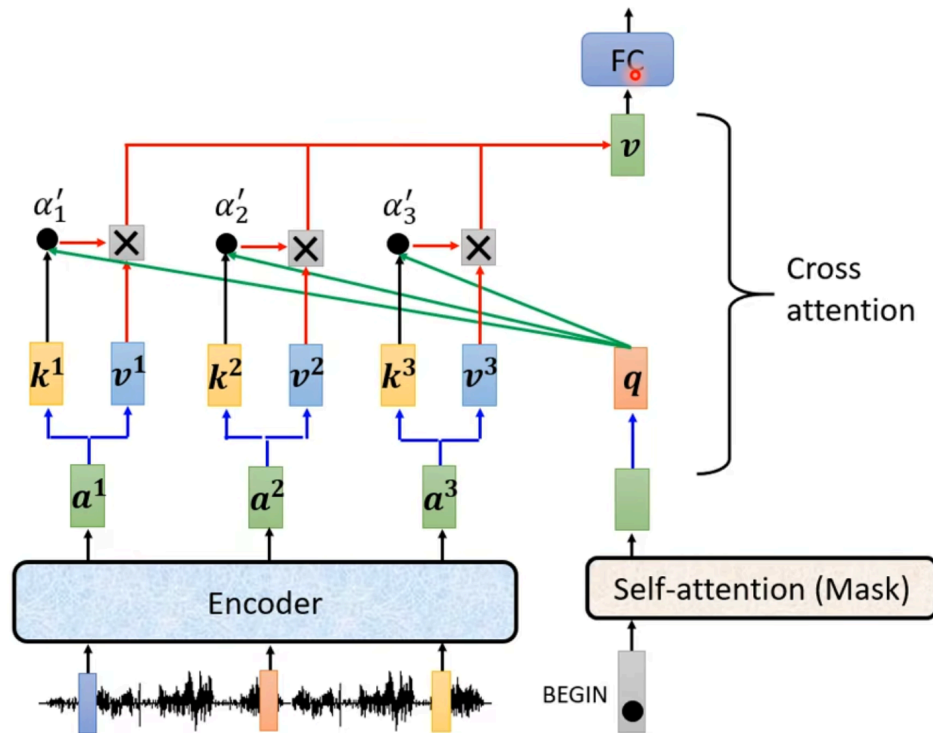
Decoder

- Self-attention $\Rightarrow$ Masked Self-attention
只能看到自己之前的输入
- 如何确认输出的停止：准备一个特殊的 **End** 符号
- Non-Autogressive vs Autoregressive

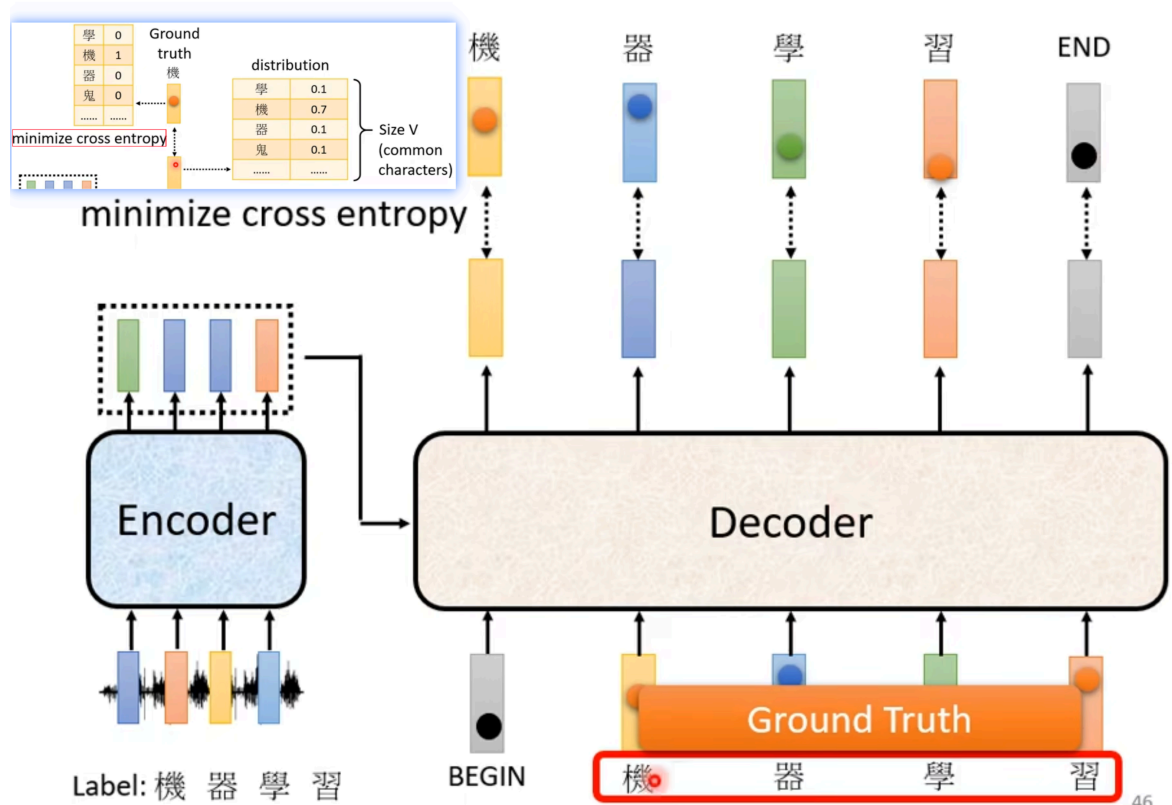
AT v.s. NAT



- 交叉注意力机制



训练



46

Teacher Forcing : Using the ground truth as input

- Copy mechanism

复现

pytorch语法相关问题

1. `Union[str, Callable[[Tensor], Tensor]]` 这个用法代表什么
2. LayerNorm具体用法和实现
3. `TransformerEncoder` 中的 `mask: the mask for the src sequence (optional)`. 这个属性是干嘛用的
4. `self.register_parameter('para', None)` 与 `self.para = None` 的区别:是否存入 `_parameter` 字典

Transformer代码细节相关问题

1. 弄清楚Feed-Forward Network的作用
 - a. FFN负责在一个独立的、更高维的空间对Attention整合的信息进行更抽象的特征提取



Attention就像一个会议室，大家交换信息；FFN就像每个人回到自己的工位，独立思考并消化刚才听到的信息，更新自己的认知，从而更好做出推理

- b. 从Attention到FFN进行了升维， $d_{\text{model}} \Rightarrow d_{\text{ffn}} \Rightarrow d_{\text{model}}$, $512 \Rightarrow 2048 \Rightarrow 2048$, FFN维度更高，更宽，从而可以更好的拟合更复杂的数据
2. 位置编码的实现？

在输入到Transformer模型前就应对数据进行处理，所以在这里不讨论
 3. `self.head_dim = embed_dim // num_heads` : `head_dim` 是啥意思

拆成头个数个子空间，相当于多通道，具体实现就是拆分最后一维(`embed_dim`), 保证计算量不变。
 4. `in_proj_weight`: 为什么要以qkv维度是否相同而分不同情况，当q、k、v维度相同时把 W_q, W_k, W_v 合并成一个矩阵？

```

if not self._qkv_same_embed_dim:
    self.q_proj_weight = Parameter(torch.empty((embed_dim, embed_dim), **factory_kwargs))
    self.k_proj_weight = Parameter(torch.empty((embed_dim, self.kdim), **factory_kwargs))
    self.v_proj_weight = Parameter(torch.empty((embed_dim, self.vdim), **factory_kwargs))
    self.register_parameter('in_proj_weight', None)
else:
    self.in_proj_weight = Parameter(torch.empty((3 * embed_dim, embed_dim), **factory_kwargs))
    self.register_parameter('q_proj_weight', None)
    self.register_parameter('k_proj_weight', None)
    self.register_parameter('v_proj_weight', None)

```

当三个变量维度相同时，可以把它们纵向拼接起来，进一步提高并行性：

D

5. query的长度就是目标输出的长度，毕竟是要把对每个query计算注意力得到相关上下文

MultiheadAttention

调用：self.self_attn = MultiheadAttention(d_model, nhead, dropout=dropout)

- init
 - self.head_dim = embed_dim // num_heads：head_dim 是啥意思
 - self.out_proj = NonDynamicallyQuantizableLinear 一个类，弄清楚这是干嘛的
- forward
 1. 搞懂Mask
 - key_padding_mask(Optional): key_padding_mask = F._canonical_mask
 - attn_mask(Optional)
 2. why_not_fast_path

pytorch常用函数

1. torch.randint
2. torch.nn.functional.pad
3. torch.unsqueeze
4. torch.cat
5. torch.stack: argument must be tuple of Tensors
6. torch.nn.Embedding：构建时 num_embeddings=max_words_num+1，也就是要预留一个<PAD>位置。本质是对句子的每个词构建了一个查表矩阵（Embedding.weight），

然后在表示句子时就可以用对应的索引来表示每个词，对于长度不够的句子空缺处就用0填充，表示用矩阵中下标为0的向量来填充。



当一个句子中的两个词相同时，它们在矩阵中对应的值也会相同，这个时候就需要对Embedding做进一步处理了，比如加入位置编码。

7. Tensor.masked_fill

TODO

✓ 实现 `MultiheadAttention` -

调用：`self.self_attn = MultiheadAttention(d_model, nhead, dropout=dropout)`

`add_zero_attn` `MutiheadAttention`的一个成员变量，弄清楚其用处

☐ 还是要弄成batch_first

☐ 完成一次完整的训练、推理的过程

✓ 审视代码，解码器的输出就直接是最后的结果了，是不是还缺少了最后的线性层和softmax

2026.1.26

已经实现self-attention了，现在需要补充mask的相关实现，从而补全cross-Attention的功能

✓ 实现 `scaled_dot_product_attention`

1.27 16:48

☐ 补充Mask，需要修改上面所有的代码，encoder、decoder等

☐ key_padding_mask (Optional)

☐ attn_mask(Optional)

2026.1.27

修正了in_projection_packed，实现了 `scaled_dot_product_attention`。关于mask，假如在声明Transformer类的时候不传入mask那后面也不会用到。

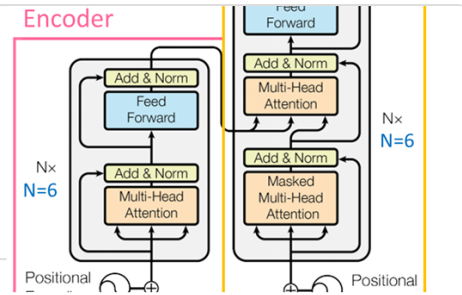
☐ 跟着逐行实现这个代码敲一遍，记录torch函数的用法

参考资料

Transformer—Attention Is All You Need

隨著深度學習的發展，自然語言處理 (Natural Language Processing, NLP) 技術越趨成熟，近年來在情感分析、機器翻譯、語音辨識、對話機器人等任務上均有很不錯的結果。

 <https://medium.com/ching-i/transformer-attention-is-all-you-need-c7967f38af14>



19、Transformer模型Encoder原理精讲及其PyTorch逐行实现_哔哩哔哩_bilibili

直播深度学习算法原理与项目源码讲解，欢迎关注我的直播间：

https://live.bilibili.com/14297368?spm_id_from=333.999.0.0, 视频播放量 234451、弹幕量 1181、点赞数 6106、投硬币枚数 5177、收藏人数 16865、转发人数 1012, 视频作者

 <https://www.bilibili.com/video/BV1cP4y1V7GF>



MultiheadAttention新手从0实现 常见错误赏析（pytorch 版）_哔哩哔哩_bilibili

建议倍速观看，代码：github: Tengfei-Ma13206/MultiheadAttention_fromScratch 欢迎 star, 视频播放量 70、弹幕量 0、点赞数 6、投硬币枚数 4、收藏人数 5、转发人数 1, 视频作者 马腾飞每天更新, 作者简介 - 2014年龙江县奥迪双钻“我是火力少年王”挑战赛亚军 -

 https://www.bilibili.com/video/BV1tpUZBYECi?spm_id_from=333.337.search-card.all.click&vd_source=985d6f4a02dd9c8bb323943cf53b89f3

